Thruput Manifesto

GEOFF RAMSEYER, ADAM CHAMBERS, CHRIS FROST, DANIEL GIFFIN,
JACK HUMPHRIES, ADRIAN LANZAFAME, GIULIANO LOSA, PETAR MAYMOUNKOV,
MARK MAYO, DAVID MAZIÈRES, VINITA PAWAR, AND RYAN STUTSMAN

1 Introduction

Digital money had better be a strict improvement over physical cash: it must be free to use, open to anyone, and entirely under the owner's control. And yet when we move payments online, we lose control of our financial resources and gatekeepers restrict access to payment infrastructure. Even though it is vastly easier to send digital messages than physical ones, transactions routinely carry hidden fees.

This is an absurd state of affairs.

Blockchains promised to fix this. Decentralization and replication mean that a blockchain keeps running even when nodes suffer hardware failure or regional networks experience outages. Smart contracts let anyone upload code governing the movement of their assets, and immediately know that their rules will be followed by the global system. The only requirement to participate in all of this is a network connection and a device capable of calculating a cryptographic signature, which any new smartphone can handle easily.

And yet, today's blockchains are slow and expensive to use. Blockchains are forced to charge increasingly high fees as demand to use them increases, because they cannot easily scale up the volume of transactions they can handle. This limitation is not a question of just improving the engineering, but is a fundamental limitation of their architectures.

Our mission, with Project Thruput, is to eliminate every practical scaling limit so that we can make smart contract execution free.

1.1 The Curse of Ordering

The root of the problem is that blockchains pretend to execute transactions sequentially. This is a natural design choice, and makes it easy to ensure that smart contract execution produces the same results on every node. Developers are used to writing sequential, "single-threaded" code.

Yet there is only so much sequential computation that can be done within one unit of time. We cannot simply buy a faster CPU core today. The common choice is to subdivide a unit of time into more, shorter slices, meaning more transactions but less compute per transaction. That limits the complexity of smart contracts and pushes developers into arcane tricks to hyperoptimize smart contract computation [2, 14, 31]. Adding CPU cores can accelerate workloads where transactions rarely modify shared state, but when one transaction depends on the result of another, the second must wait for the first. Yet shared state is central to the financial markets that blockchains enable. Blockchains regularly see many users transacting on the same market, or many buyers paying one merchant, or moving assets through the same DeFi protocol.

Additional CPUs fundamentally cannot provide more than a bounded speedup on these ordered workloads. Boosting throughput therefore requires increasingly sophisticated engineering projects in pursuit of ever-diminishing marginal returns. Imposing artificial divisions on state, as in sharded

Author's Contact Information: Geoff Ramseyer, Adam Chambers, Chris Frost, Daniel Giffin,
Jack Humphries, Adrian Lanzafame, Giuliano Losa, Petar Maymounkov,
Mark Mayo, David Mazières, Vinita Pawar, and Ryan Stutsman.

blockchains and Layer-2 systems, merely hides the problem of managing shared resources (cross-shard transactions become even more expensive) and makes user experience meaningfully worse. No user should need to think about what shard their money lies on.

As a result, what a sequential blockchain delivers is a system that is expensive to use, can barely realize the full potential of smart contract customization, and cannot support concurrent usage by everyone around the world.

1.2 Project Thruput

Thruput demonstrates that this assumption, that blockchains must totally order every transaction, is false. In Thruput, blocks remain ordered relative to each other, but all transactions within one block semantically execute at **exactly the same logical instant**. Instead of optimistically parallelizing sequential work, Thruput's implementation executes transactions in a totally unpredictable, unspecified, and interleaved physical order.

And yet, at the end of each block, all Thruput nodes agree on the state of the blockchain and what each transaction did. This deterministic execution follows not from ordering but from structured APIs to read and write persistent state that make concurrent transactions *commute* with each other: updates produce the same aggregate result regardless of the order in which they are applied. Blocks are constructed carefully to preserve key invariants, like non-negative token balances. dm marked this as hard to follow; is the above better? Contracts encapsulate these APIs behind familiar interfaces, and can recover ordering between operations as necessary.

This design enables Thruput to scale up in the simplest way possible: by adding additional CPUs to process transactions [10]. Moreover, our implementation scales **linearly** to hundreds of CPU cores. Doubling the number of CPU cores doubles the throughput. This design fundamentally changes the tradeoff between blockchain throughput and capacity planning for users of the chain. We do not claim any level of maximum throughput; instead, our design means one can calculate, for any given level of desired transaction throughput, how many CPUs are needed.

Furthermore, this design enables new system architecture opportunities in a Thruput node. Each node is implemented as a distributed system, with transaction execution on different machines from the database. There are a limited number of CPUs that can fit on one motherboard, so Thruput scales instead by connecting additional commodity machines. Thruput can tolerate database I/O delays because Thruput's unordered design means that while one transaction might be stalled waiting for a read from the remote database, the CPU always has more work available to do.

Through all of this, Thruput acts from the outside like a normal blockchain. Each node in the system maintains a copy of the chain. There is no sharding of state or artificial subdivision between groups of users, and no extra security requirements beyond standard honest-supermajority assumptions of consensus protocols. Thruput does not impose additional requirements, such as precalculated transaction read/write sets, on end users.

This scalability lets Thruput realize the vision of a global, open, free, and programmable financial infrastructure. Because Thruput is not bound by any fixed level of transaction throughput, Thruput can scale supply arbitrarily in response to demand, and in so doing obviate the need to charge any transaction fee whatsoever to limit demand. Extreme events in one part of the world or bursts of activity in a memecoin do not crowd out normal payment activity (§8). There will, of course, be plenty of valuable services (which might themselves charge fees) built on top of freely transferable digital money, but the smart contracts themselves will be free to run.

Project Thruput is under active development. It extends the research of some of the authors [29] into a fully distributed system architecture and a vastly more capable system. We describe here the technical innovations that we believe will make Thruput the highest-throughput blockchain possible, outline the current state of development, and survey our technical roadmap.

2 Thruput Smart Contract Overview

Like any blockchain, smart contract execution on Thruput must be entirely deterministic and thus replicable across every node in the system. Thruput's core innovations lie in parallelism and the tools for smart contracts to manage concurrency, so in all other aspects of the system design, we strive to make Thruput as "normal" as possible.

Smart contracts in Thruput are implemented as RISC-V programs (`riscv32-im`). Memory and registers in the program state are discarded after the program finishes, and contracts read and write to persistent storage via a system call interface. Persistent storage is a key-value store with 64-byte keys and values with structured types, discussed below section number. Each contract is deployed at a 32-byte "ContractAddress" and owns a 32-byte keyspace within the key-value store (so every item is indexed by 'ContractAddress || StorageKey'). One contract can synchronously invoke another contract within the same transaction. Contracts emit events to signal actions to external systems.

What makes Thruput unique lies in how transactions interact with that key-value store. In a traditional, sequential blockchain, values written in one transaction can be read in the next transaction, and in fact maintaining coherence about what writes are visible to what reads is the core engineering challenge when optimistically executing a sequence of transactions in parallel (via techniques like Block-STM [17]). Thruput has no need for a complex concurrency management protocol. Instead, Thruput changes how transactions read and write to shared state.

Snapshot Isolation. All transactions in a Thruput block read from the same snapshot of the database, as it existed at the end of the previous block. Each transaction's side effects cannot be observed by any transaction in the same block, and are only observed by transactions in future blocks.[1] These semantics are similar to Snapshot Isolation in traditional database designs.

Write Contention. If every transaction wrote to distinct keys, then of course Thruput would have no problem reconciling concurrent writes to produce the database snapshot for the next block. The interesting use-cases for blockchains, however, are exactly those where contention on shared data is frequent and inherent. Markets, and especially automated market-makers, sequence trade orders and offer the best price to whichever trade is executed first. NFT drops often allocate a limited number of objects to the first buyers. Even simple payments experience contention on high-activity accounts, such as flows in and out of exchanges and banks, or payments to the tax authorities on tax day.

What Thruput does is ask contract writers for a little bit of additional information on how to resolve concurrent writes. Take a standard token contract as an example. In a standard blockchain environment, to send a payment, the contract reads the sender's balance, subtracts from it, aborts if that balance is negative, and writes an updated balance. It then reads the receiver's balance, adds to it, and writes an updated balance (and performs other authorization checks, executes callbacks, etc). The blockchain node executing that smart contract, however, just sees an opaque sequence of bytecode operations, along with two reads and two writes to the key-value store, and has no knowledge about the algebraic relationship between the reads and the writes.

Thruput, instead, asks contracts to surface that small amount of information to the runtime, so that the blockchain can resolve concurrent writes in a predictable manner. Instead of just writing an updated balance (which, to the runtime, looks like an opaque string of bytes), the token contract outputs to the runtime an instruction to "add X" to the value (whose type is an integer, as implied by writes to that key in prior blocks) at one key, and "subtract X" from the value at another key. Then, when multiple transactions modify the same balance (if, for example, one user sends or

[1] With limited exceptions; see §9.

receives many payments at once), the runtime can add all of the additions and subtractions to the balance to compute the value for the end-of-block snapshot. Every node will agree on that end-of-block snapshot because addition is commutative. That commutativity allows Thruput to be a deterministic blockchain without suffering any of the synchronization overhead of keeping CPUs executing in a predictable order.

Figure 1 illustrates how a token contract can encapsulate these primitives behind a familiar `transfer` API. Other contracts can use ordinary synchronous contract calls.

Commutativity is, however, insufficient to actually write useful applications on a blockchain.[2] Balances of a token, for example, cannot be allowed to become negative. Thruput therefore enforces constraints on the aggregate behavior of transactions in a block. In the case of a token balance, Thruput guarantees that after adding up all of the additions and subtractions to a balance, that integer's value is nonnegative. Thruput achieves this by selectively including transactions when constructing a new block proposal. Transactions that would violate a constraint are deferred to a future block @dm is this better?.[3] The block proposal process, despite the invariants that must be maintained, is no more expensive computationally than replaying a proposal from another Thruput node; see §3 for details.

In other words, each value type (detailed in §5) in Thruput has two associated properties.

Typed Commutative Interface. Contracts read and modify values via a structured API. Each modification (like "add" to an integer) is associated with a particular type of value. These structured modifications allow the Thruput runtime to mediate writes across multiple transactions.

Value Invariants. The Thruput runtime maintains specific invariants (defined by the value type) on modifications to a value by transactions in the same block. These prevent the kinds of semantic conflicts, like a balance going negative, that cannot be addressed within one transaction, in Thruput's parallel state model.

These tools are optional for contract developers. Thruput does not force concurrency onto contracts, but gives contracts the tools to manage it should they choose. A contract writer could choose to ignore them (and use only a basic "bytestring" type), at which point Thruput would not have information about how to resolve concurrent writes and would have to exclude all but one transaction on the contract per block, in the worst case. In the other extreme, contracts using these tools can handle extreme, worst-case contention, where for example millions of users send a payment to one address all at once. There is furthermore a contract-agnostic transformation (RCQ ref) that allows any contract that needs strictly sequential semantics to recover those semantics, as necessary. Concurrent contracts run at arbitrarily high throughput, strictly sequential contracts run at the speed of one CPU (the best we could hope to promise), and neither slows down the other.

Similarly, because every contract modifies state via a set of structured system calls, the rest of a smart contract can be arbitrary, untrusted bytecode. Thruput requires no static analysis or approval process to guarantee commutativity. Like any other blockchain, smart contracts on Thruput are uploaded by untrusted users and may be actively malicious. A bug in one contract cannot corrupt the rest of the chain.

[2]Without these invariants, the Thruput state model would look like an application of Conflict-Free Replicated Data Types (CRDTs) [33]. A CRDT cannot directly enforce a non-negative balance, because a replica subtracting from a balance cannot know whether other, not-yet-applied operations also subtract from it. Escrow-based variants [5] sidestep that limitation by partitioning the balance among a fixed set of replicas ahead of time and having each replica refuse subtractions it cannot locally cover, which is akin to sharding the blockchain across a fixed, finite set of subnetworks. By contrast, Thruput needs only to reconcile a finite set of transactions in one block, so it can admit subtractions on demand from a single reservation table and defer the rest.

[3]The decision of, for example, which of a pair of conflicting transactions to exclude is up to the proposer (i.e. nondeterministic), but block execution is deterministic once a node stamps a set into a block proposal.

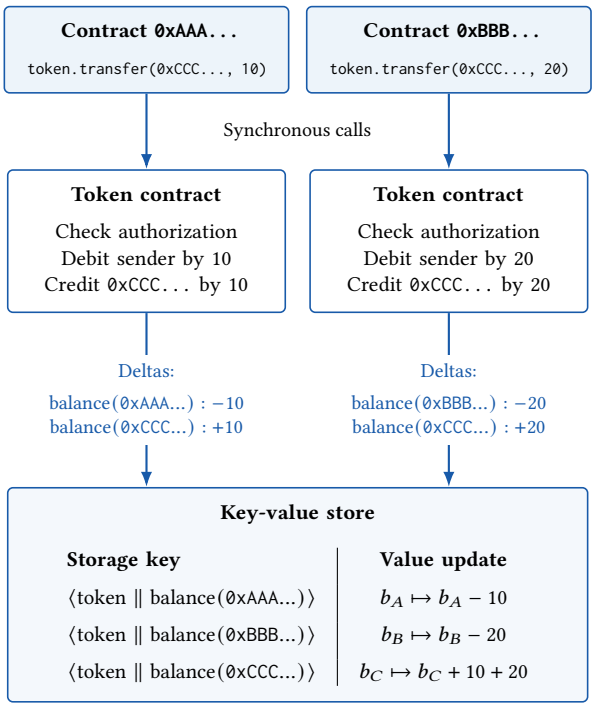


Fig. 1. **Two transactions in the same block.** The token contract wraps integer update calls in an ordinary `transfer` API, which other contracts can synchronously call. Two transactions (left and right) both modify the balance of address `0xCCC...`, and the Thruput runtime mediates the concurrent access.

3 Proposing and Validating a Block

Thruput's state model enables it to execute all transactions in a block in parallel with virtually no synchronization overhead between transactions. This parallelism holds regardless of the frequency that concurrent transactions access overlapping state, and holds regardless of whether Thruput is constructing a new block proposal or validating a proposal from another node.

Executing a transaction proceeds in two phases. First, each transaction starts with an instruction to invoke the contract at some address with some input calldata. A logical thread[4] loads the contract from the database and begins interpreting the RISC-V instructions there. As that proceeds, that contract may sometimes invoke various system calls, which might read additional values from the database or create requests to write values back to the database. A system call may also request to invoke another contract. Contract invocations that fail (i.e., run out of gas or access out of bounds memory) in this phase become no-ops. All writes are buffered locally in that transaction; a transaction can read values that it wrote earlier, but can never read values written by concurrently-running transactions.[5]

Eventually, that top-level contract invocation comes to an end. The second phase of transaction execution is a reserve-commit protocol (like two-phase commit) to apply writes to the database. For each modified key, the transaction first `reserve()`'s the right to apply each modification to the database. In a standard two-phase commit process, this would lock the entire key-value pair.

[4]Each transaction runs as a C++ coroutine, not a thread, so one thread might run several transactions at once.
[5]With limited exceptions; see §9.

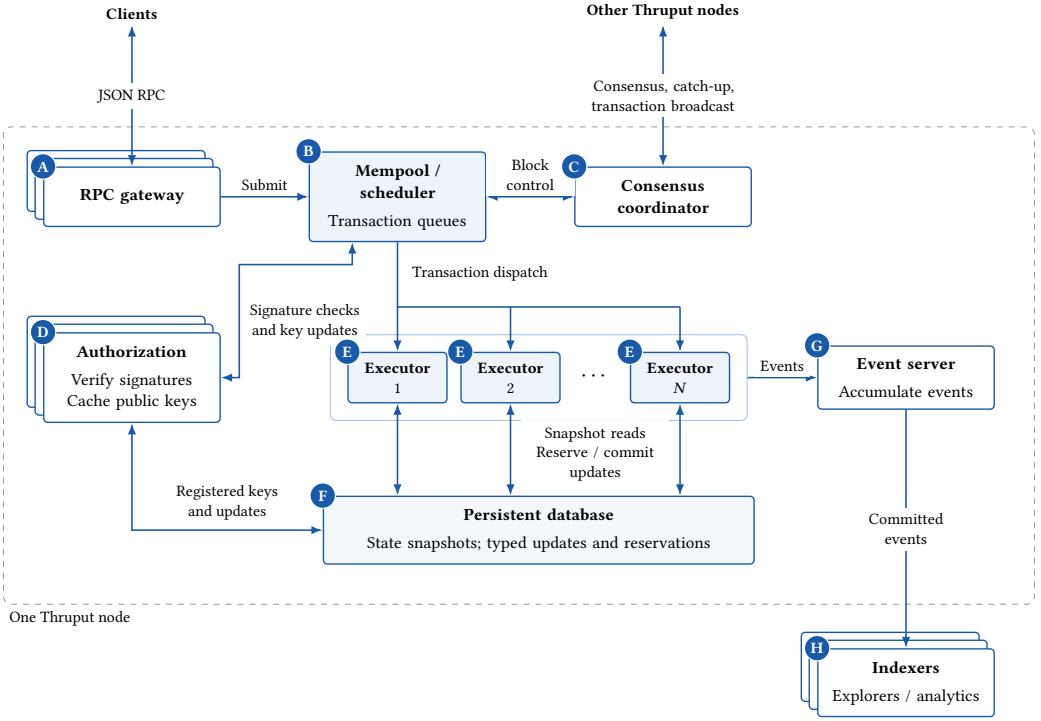


Fig. 2. Data flow within one Thruput logical node.

Thruput uses a fine-grained locking scheme, depending on the type of the value. For example, for a token balance (non-negative integer, ref) with snapshot value of 100, Thruput allows up to 100 units worth of subtractions to be `reserve()`d at the same time. If all reservations can be acquired successfully, the transaction then `commit()`s each of its writes.

Nodes in Thruput sometimes are elected leader, to propose a new block, and sometimes validate a block proposal from another node. When proposing a block, a Thruput node runs the above process and greedily packs in any transactions that succeed at reserving all of their storage updates. Transactions that fail to acquire reservations are excluded but can be retried in future blocks.

Validating a proposal means executing all transactions and checking that each transaction could acquire all of its reservations. Block validity is a property of the set of transactions, not the order in which they touch the database. Consensus rejects proposals that fail validation. The reserve-commit process guarantees that no honest proposer can accidentally produce an invalid block, and guarantees that an honest block proposal satisfies the type-specific invariants for all values in persistent storage.

4 System Architecture

Thruput is built from the ground up to be a distributed system, where the key components can independently scale. Figure 2 outlines the architecture of one node.

External RPC Gateways (A) mediate access from external clients, who might send transactions via a JSON API or subscribe to a websocket feed of events. Transactions are forwarded to a mempool process (B), which buffers pending transactions locally. A consensus coordinator (C) manages communication with other Thruput nodes, handles consensus messages, and answers requests for

past blocks from nodes catching up to the chain. It also manages a multicast broadcast subsystem to forward new transactions from the mempool to other Thruput nodes.

Adjacent to the mempool is an authorization service (D), which is responsible for the (expensive) cryptographic signature verification on each transaction. Note that Thruput has no notion of "Externally Owned Account"; instead, every address can configure a set of public keys that can authorize transactions to start with an invocation of the contract at that address (§6). This means the authorization service must load these registered keys from the persistent database (F). The authorization service is stateless, but maintains a cache of recently used public keys. The mempool stores transactions that pass authorization on a queue of transactions that are ready to execute. When a block of transactions commits, some addresses might have changed their registered public keys, so the database notifies the authorization service and the mempool of all addresses with changed keys that must have their authorization re-checked and cache entries invalidated.

When a Thruput node is called upon to propose a new block, the mempool dispatches transactions from its runqueue to transaction executors (E). These stateless services execute transactions as they arrive, reading and writing to the database as necessary. Thruput's snapshot-read model conveniently allows the executors to trivially cache values read from the database. This design is feasible in large part because of the Thruput execution model. When a transaction reads from the database, that access reads data not in local memory but on a remote machine, incurring nontrivial latency. But during that remote access, the executor's CPU can simply switch to executing another ready transaction, so CPU time is never wasted. In a standard sequential chain, by contrast, that remote access could stall every other transaction in the block.

The mempool prioritizes which transactions to include according to a scheduling policy, related to the number of native coins held at each address and how recently those coins were used to schedule a transaction (see later ref). When a block of transactions commits, the database notifies the mempool of native coin transfers, to inform this scheduling policy.

Transactions may emit events, which are accumulated separately on an event server (G). When a block commits, these events are forwarded to downstream indexers (H), which might power block explorers or other analytical services.

5 Concurrent Data Types

We outline here the data types and APIs which Thruput smart contracts use to interact with persistent state. This is not a complete API reference, and we may add additional types in the future, as needed. We also provide, for example, system calls to emit events, to query what address called the function ('msg.sender'), and to verify a signature, among other functionality.

We design the Thruput semantics to resolve as many conflicts as logically possible by default. However, any contract that needs exclusive access to a particular value can add an exclusive lock flag to any storage write.

5.1 Bytestrings

The minimal, default way for a contract to write data is as an opaque bytestring. This is how most blockchains operate. Yet this gives no semantic information to the Thruput runtime to resolve writes to the same key. Without any information, Thruput cannot do anything but reject transactions that write different values to the same key in a block. By default, Thruput admits pairs of transactions that write the same string to a key. This behavior can be overridden by the exclusive lock flag.

Configuration data, like the address of an administrator on a contract, fits well in this model and has no meaningful notion of commutativity to express to Thruput. One might use the exclusive lock flag when initializing a new contract, to ensure that stateful operations associated with the initialization (like transferring funds) are only performed once. In other settings, such as computing

the closing time of an auction and updating a state variable to mark an auction as closed, multiple transactions can make that write and proceed concurrently.

5.2 Nonnegative Integers

Financial applications, like token balances, need some notion of a balance that cannot be negative. For these cases Thruput provides a "non-negative integer". The natural API for a token balance would be to add and subtract to it; Thruput gives a generalized version of that API, which we call a `nnint_set_add`(X, Y) operation. X and Y are any two *signed* 128-bit integers. Logically, this operation sets the value of the integer to X, and then adds Y to that base value. The corresponding `nnint_get` API returns the most recently written (X, Y) pair on that key, or the snapshot value (and a 0 adjustment on it) for values not yet modified in that transaction. This API can implement both token balances and more general linear constraints on financial operations, like collateralized borrowing limits and spending velocity constraints.

Constraint: Nonnegativity. Two concurrent transactions that produce `nnint_set_add`(X_1, Y_1) and `nnint_set_add`(X_2, Y_2), respectively, conflict if $X_1 \neq X_2$ (analogously to the bytestring case). A collection of operations `nnint_set_add`(X, Y_1), ..., `nnint_set_add`(X, Y_n) is invalid (i.e. reservations of these operations cannot all succeed) if $X + \sum_{i=1}^{n} Y_i < \min(0, X)$; that is, if the value of the integer becomes negative. In the interest of maximizing flexibility for contracts, we allow $X < 0$ (as contracts can choose to abort before making a `nnint_set_add` system call with such an argument), in which case the constraint means that the value cannot become further negative. Thruput also enforces an analogous constraint against overflowing the maximum value of a 128-bit integer. 128 bits is an arbitrary choice, but we believe it provides a reasonable tradeoff between arithmetic width and implementation complexity.

5.2.1 Example: Token Contract. A token contract can represent token balances and allowances as non-negative integers. A transfer means reading a user's snapshot balance, and then outputting an `nnint_set_add()` operation, using the observed balance as the base value. Figure 3 sketches pseudocode for part of an ERC20-style interface. We emphasize that other contracts calling the transfers can call the methods as synchronous cross-contract calls, without knowing how the token is implemented.

5.2.2 Example: Transfer Velocity Limit. The base value portion of the `nnint_set_add()` operations lets the operation extend beyond token balances. A transfer velocity limit, for example, can be implemented as a non-negative integer whose base value resets on a regular frequency.

Figure 3 shows a wallet that permits at most `LIMIT` tokens to be transferred during each window of `WINDOW` blocks. At a window boundary, concurrent calls write the same epoch marker and reset the budget to the same base value; their spending deltas combine. The budget update and token transfer happen within one transaction.

5.2.3 Example: Collateralized Lending. A collateralized borrowing/lending protocol (such as Compound [12], Blend cite, Aave cite) allows users to post collateral (typically some volatile asset) and borrow another asset against it (typically a stable asset, like a USD-pegged stablecoin). The amount that can be borrowed is a complicated calculation which might include pricing the user's posted collateral with various protocol-defined discount factors based on, for example, asset volatility. Interest rates similarly float (typically) in response to a variety of factors, such as the amount of an asset posted as collateral relative to the amount of that asset borrowed.

To implement such a protocol on Thruput, a contract would compute updates to interest rates and collateral valuations once per block. Compound already uses a related pattern on a sequential chain: its interest indices accrue only when the block timestamp advances [11]. Thruput can then

Token Implementation

```
def transfer(to, amount):
    require 0 <= amount <= I128_MAX
    sender = msg.sender
    _adjust(balance_key(sender), -amount)
    _adjust(balance_key(to), amount)
    emit Transfer(sender, to, amount)
    return True

def balanceOf(owner):
    key = balance_key(owner)
    base, delta = nnint_get(key)
    return base + delta

def _adjust(key, change):
    base, delta = nnint_get(key)
    delta += change
    require (base + delta) >= 0
    nnint_set_add(key, base, delta)
```

Pseudocode; a partial ERC20cite implementation

Transfer Velocity Limit

```
def spend(to, amount):
    require msg.sender == owner
    require 0 <= amount <= I128_MAX
    epoch = block_height() // WINDOW

    if bytestring_get("epoch") != epoch:
        bytestring_set("epoch", epoch)
        base, delta = LIMIT, 0
    else:
        base, delta = nnint_get("budget")

    delta -= amount
    require base + delta >= 0
    nnint_set_add("budget", base, delta)
    require token.transfer(to, amount)
```

Pseudocode; a fixed-window spending cap. `owner`, `token`, `LIMIT`, and a positive `WINDOW` are configured at deployment. A missing epoch marker triggers the first reset.

Fig. 3. Two uses of concurrent integer updates. Left: an ERC20-style token interface. Right: a wallet enforcing a transfer limit per window. The runtime checks aggregate balance and budget constraints across transactions, while contracts expose ordinary synchronous methods. Pseudocode omits encodings and arithmetic overflow checks.

implement borrowing headroom and total debt as non-negative integers. Figure 4 sketches a lending market's borrow and repayment paths. The contract computes the borrowing limit available to a user, minus its existing debt, and uses that margin as the base value of the `nnint_set_add()` call.

These loans can become undercollateralized, when, for example, the value of the collateral asset crashes. Each protocol has some mechanism for absorbing those losses; that choice is not especially relevant here. What matters for Thruput, though, is that these undercollateralized states are still representable, so that contracts can make forward progress to resolve these cases. This setting motivates why we allow the base value of `nnint_set_add()` to be negative.

In other words, Thruput enforces only the constraints that cannot be enforced by a transaction itself. One transaction cannot directly observe concurrent operations from other transactions. Instead, contracts express those invariants to Thruput so that the runtime can enforce them. However, one contract invocation can observe states written in prior blocks, so Thruput places as few constraints as possible on how contracts operate on past state.

5.3 Queues

Some contracts, such as an automated market maker or an orderbook[6], require strictly sequential semantics. For these contracts, Thruput provides a multi-producer, single-consumer queue. This queue allows the contract to recover limited ordering on its own operations, without forcing the whole chain into a single total order.

[6]A decentralized exchange can be implemented in Thruput's concurrency model, but this requires substantial changes to the trading mechanism [30], which touches on long-running open questions in the economics literature citations

Borrowing and Repayment

```
def borrow(amount):
    require 0 < amount <= I128_MAX
    user = msg.sender
    _change_debt(user, amount)
    require token.transfer(user, amount)

def repay(amount):
    require 0 < amount <= I128_MAX
    user = msg.sender
    _change_debt(user, -amount)
    require token.transferFrom(
        user, self.address, amount)
```

Shared Borrowing Constraint

```
def _change_debt(user, change):
    dkey = debt_key(user)
    mkey = margin_key(user)

    debt, delta = nnint_get(dkey)
    delta += change

    margin = snapshot_limit(user) - debt

    # Debt cannot be negative
    nnint_set_add(dkey, debt, delta)
    # If margin < 0, debt can only be repaid
    nnint_set_add(mkey, margin, -delta)
```

Snapshot helpers ignore this transaction's pending writes; `delta` accumulates its debt changes. The borrowing limit values collateral using the block's fixed prices and discount factors.

Fig. 4. Collateralized borrowing as a concurrent linear constraint. Interest accrual calculations omitted.

Constraint: Unique Consumer. A caller of the contract can insert a message into a queue. Insertions do not conflict with each other, and the queue is FIFO-ordered across blocks (and lexicographically sorted within a block). One transaction can then consume entries from the queue one by one, and only one transaction is allowed to consume from the queue in a block. This means that unless the queue is explicitly deleted, every entry that is placed into a queue can be consumed at most once, and entries are never dropped.

5.3.1 Example: AMM Contract. An automated market-maker such as Uniswap [1] maintains reserves of two assets and allows users to exchange either asset for the other, at an exchange rate that adjusts based on the amounts of assets in the AMM. The price calculation formula is different in each implementation, but generally the price changes after every trade; as such, trades generally do not commute with each other, and the AMM needs to execute transactions in a strictly sequential order.

Thruput implements this primitive by wrapping calls to the AMM in a queue. Callers submit trade requests to the queue, and then a separate transaction "turns the crank" to pop elements from the queue and apply them to the AMM. cite people doing this notion of cranking on solana Figure 5 gives a simplified example using Uniswap v2's [1] pricing rule.

There are several limitations to this design. The first is that unlike the rest of Thruput, there is a limit to the number of transactions that a single AMM can handle in a block. This means that an AMM may wish to charge a transaction fee (using e.g. an EIP-1559 style dynamically adjusted fee) to limit the number of enqueued swaps to a rate that can be sequentially processed. In other blockchains, this fee mechanism is handled by the consensus protocol; here, it must be implemented (but the fee revenue can then be paid directly to the liquidity providers of the AMM). Second, calls into an AMM cannot be completed synchronously by other contracts, which limits what can be done within a single transaction.

Submit a Swap

```
def swap_x_for_y(amount, min_out):
    require amount > 0 and min_out >= 0
    user = msg.sender

    queue_insert(SWAP_KEY,
        (user, amount, min_out))
```

Process Queued Swaps

```
def crank():
    x, y = bytestring_get(RESERVES)
    amm_address = get_current_address()
    for _ in range(MAX_BATCH):
        request = queue_consume(SWAP_KEY)
        if request is None:
            break
        user, amount, min_out = request

        require token_x.transferFrom(
            user, amm_address, amount)

        # Uniswap v2's 0.3% swap fee
        net = 997 * amount
        out = (net * y) // (1000*x + net)
        if out == 0 or out < min_out:
            # Refund
            require token_x.transfer(
                user, amount)
            continue

        x, y = x + amount, y - out
        require token_y.transfer(user, out)

    bytestring_set(RESERVES, (x, y))
```

Fig. 5. A simple Uniswap v2-style swap operation. Request failure handling is omitted; see §9.1.

And finally, synchronous calls that the AMM makes (such as token transfers) might still conflict with other, concurrent transactions. An AMM could handle this by transferring the token sold when the request is enqueued; then, assuming the token contract actually satisfies the token contract specification, the work of processing one trade would not depend on whether the trader still has sufficient funds at the time the request is executed.

However, that design requires case-by-case analysis and may be difficult to implement in practice. Thruput also introduces a general purpose mechanism, which we call commit locking (§9.1) to allow contracts to capture a few limited bits of information about concurrent transactions. With commit locking, the queue crank operation is a general purpose mechanism for any sequential contract, which requires no specific assumptions or case by case analysis.

5.4 Replay Nonce Set

Thruput prevents transaction replay with a (nonce, expiry height) replay tuple on each transaction. Thruput will never commit two transactions from the same contract address with identical replay tuples, but does not require that nonces on an address increase sequentially. Allowing gaps in nonces lets Thruput execute many transactions starting at some address in the same block. Consumed replay tuples are garbage collected when the chain reaches the expiry block height.

We intend to give contracts the ability to manipulate sets of nonces directly. One might, for example, have a separate notion of replay nonce for so-called "intent" mechanisms (such as [25]), but the implementation is not yet ready.

5.5 Roadmap: Other Data Types

One can imagine a number of other potentially useful data types for contracts in the Thruput model. Our early research prototypes ([29]) combined replay nonces and queues into an "ordered set" primitive. SPEEDEX [30], which operates in an analogous concurrency model, used a notion of a bitmap to prevent transaction replay, and used a specialized ordered set to implement an orderbook.

The first version of Thruput supports these data types because we believe them to be sufficient to implement a wide variety of real-world applications. We may add additional types, however, in response to developer feedback.

6 Addresses and Authorization

Computation on Thruput is so widely available that we have no need for so-called "externally owned accounts"; instead, every user interacts with the chain via a smart wallet. Each address can register one or more public keys, which Thruput uses to authorize transactions that start with an invocation on that address.

This design has the added benefit of not locking Thruput into any one signature scheme. Given that fault-tolerant quantum computers are plausibly on the horizon and that artificial intelligence is greatly assisting in cryptanalysis [26, 35], it would be imprudent to launch a new blockchain without making it trivial for users to switch between cryptographic schemes. Thruput lets users keep their existing addresses and just register additional keys (and revoke old ones). Every part of the system that interacts with a public key or signature is abstracted over the cryptographic scheme, so contracts generally do not need to think about what schemes are available.

Thruput currently supports Ed25519 keys (for backwards compatibility with existing wallets) and ML-DSA-65 (FIPS 204), but we plan to update this list as additional schemes are standardized and studied.

7 Database Design

8 No-Fee Transaction Scheduling

9 Nondeterministic Proposal Data

There are two limitations with Thruput's handling of sequential contracts via queues, as outlined in §5.3. First, the queue in Figure 5 cannot handle potential storage conflicts from concurrent transactions, especially if the sequential contract calls other contracts. And second, the queue design introduces an at least one-block delay between when an operation enters the queue and when it is applied. Thruput addresses both limitations by allowing some nondeterminism in how a block proposer executes transactions. This nondeterminism is captured as a string of bits and included in the block proposal, so that every node can replay that same nondeterminism.

9.1 Commit Locking

When one contract calls another contract, the caller must be able to catch and handle errors that occur in the called contract. The other contract could be untrusted and malicious, but a contract developer must be able to know that one bad actor cannot unilaterally shut down their application.

This kind of error handling is straightforward in a sequential blockchain. When the callee contract hits an error, control flow always returns to the caller, and the callee's state changes are

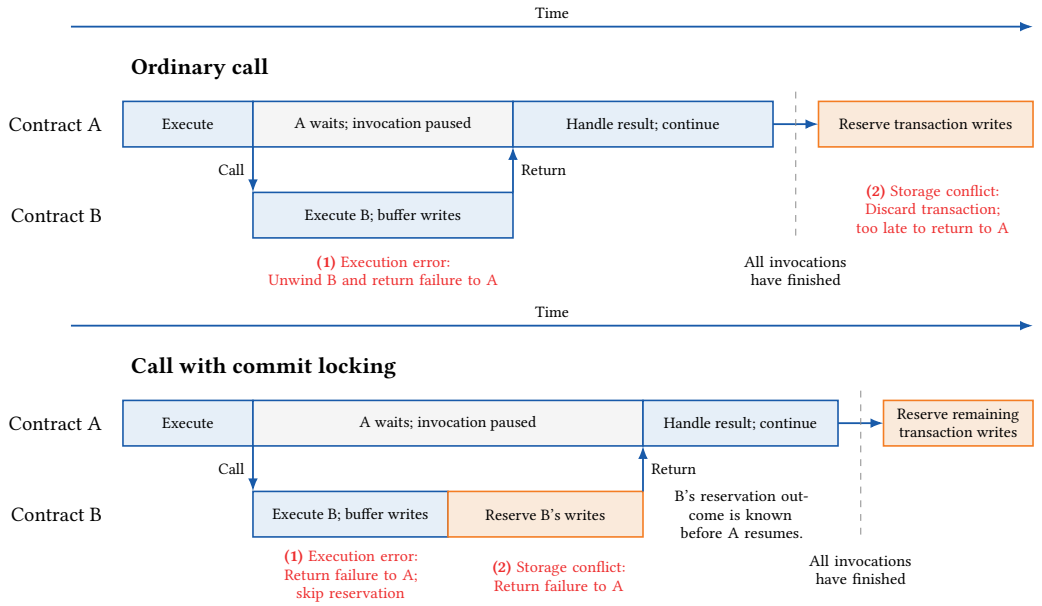


Fig. 6. Commit locking lets the caller handle both execution errors and storage conflicts before continuing. The callee's reservation outcome is logged as one nondeterministic bit for replay.

typically unwound. From that point onwards, nothing that the callee contract did can affect the rest of that transaction execution, unless that contract is called again.

However, in Thruput, a contract can encounter two distinct types of errors. The first are those that occur during execution (case 1 in Figure 6), like the above scenario, and which Thruput handles in the same manner. But second, a callee contract might write data to storage, which could conflict with a concurrent transaction (case 2 in Figure 6). But that storage conflict is not known until execution finishes, when Thruput tries to `reserve()` every storage modification. At that point, all contracts invoked in the transaction have finished executing. As such, an error there cannot be handled programmatically, so Thruput discards the transaction. This opens a denial of service vulnerability for sequential contracts using the queue mechanism of §5.3, as any call to another contract could cause the entire queue crank transaction to be discarded.

Thruput addresses this limitation by allowing a caller contract to perform all of a callee contract's storage reservations immediately when the callee finishes, instead of at the end of the transaction. This pulls forward the time at which a conflict is observed, so that the caller can observe the conflict and handle it. Failed reservations here revert all the state changes made by the callee, just as though the callee hit the first type of error. We call this mechanism "commit locking". If a caller invokes a callee with commit locking, then the caller observes one nondeterministic bit stating whether or not all of the callee's storage operations could be reserved at that time.

The queue crank in Figure 5 can invoke each request handler with commit locking. If a handler fails, the crank can retain the queue consumption and continue to the next request. A user withdrawing their funds can cause their own swap to fail, but cannot force the entire crank transaction to be discarded.

This bit is not deterministic and not externally verifiable. A malicious proposer could, in principle, pretend that the reservations failed when they could have succeeded. This mechanism is opt-in for

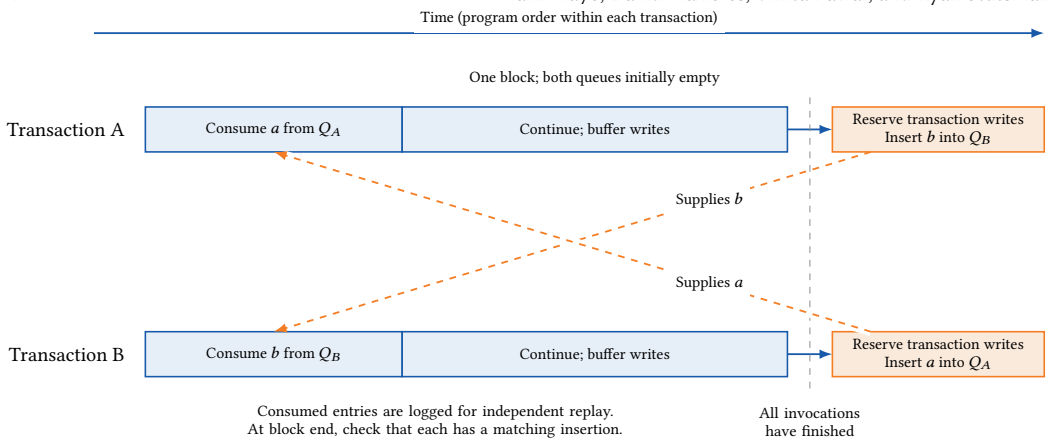


Fig. 7. A circular queue dependency allowed in Thruput. No causal execution order is required.

contract developers, and we believe best used only in limited cases (like the queue crank loop), but contracts are free to make their own choice.

9.2 Retrocausality

The other limitation of queue-based operation sequencing is that operations are delayed by at least one block, as transaction writes are not visible to concurrent transactions in Thruput. Thruput therefore allows queues to be what we call *retrocausal*; that is, we use the nondeterministic data bundled with a block proposal to allow a queue insertion to be visible to a queue consumer in the same block. Thruput enforces no global causality or sequential consistency constraints on these queues, as that would bring Thruput back to the sequential blockchain model and break its performance characteristics. The queue guarantees that each message is consumed at most once and is not lost.

This means that queue entries can be delivered in orders that are not sequentially consistent, and might even contain circular dependencies. A queue entry could go "back in time" in a transaction (hence "retrocausality"), as illustrated in Figure 7. In practice, each block proposer running the default Thruput software cannot produce such orderings, but that is not checked when validating a block proposal.

This retrocausality is implemented by logging queue entries consumed from the current block (the retrocausally consumed entries) in a transaction's nondeterministic data, bundled with a block proposal. This enables Thruput, when validating a block proposal, to replay that transaction using just the logged entries, without waiting on any concurrent transactions to finish validating and without waiting on shared data structures (so throughput is not harmed). Consistency of each queue, meaning that entries that were retrocausally consumed were actually inserted, can be checked only once at the end of the block.

10 Roadmap: External Calls

Section 9 highlights the flexibility that a small amount of nondeterministic data recorded in a block proposal provides. This flexibility, combined with Thruput's unordered execution, can give Thruput contracts a capability not viable on any other blockchain. Contracts can make **remote procedure calls** to services outside of the blockchain, all within a single, atomic Thruput transaction. To be clear, the infrastructure to support this functionality is not yet implemented. The research prototype

[29] had a limited form of this capability, but Thruput is still working through the exact design for the API to provide to contracts. There are also security concerns.[7]

Suppose that a contract initiates an RPC to some service off of the blockchain. This involves a network roundtrip over the public internet, which might easily take tens or hundreds of milliseconds, and might or might not return at all. On a standard, sequential blockchain, this would be a complete non-starter of a design, as the entire chain would be forced to stall for this single call. However, in Thruput's unordered execution model, while one transaction is awaiting a pending remote call, a node's CPUs can simply work on running other transactions until that RPC finishes. And while we cannot rely on an untrusted off-chain service to provide repeatable answers, we can bundle the call's observed response within the nondeterministic data in a block proposal, so that every node can replay the RPC without actually making the call.

There are of course limitations in this design. Any service that wishes to reliably talk to the chain must have server(s) with low-latency connections to most if not all Thruput nodes. The RPC responses would need to be self-authenticating (for example, the response could be signed by a key that is preregistered with the caller contract). And each RPC has an inherent maximum timeout, as we cannot wait for a transaction for longer than Thruput waits for each block.

Of course, asynchronous calls to external services can be implemented on today's blockchains across multiple blocks, but not within the same, atomic transaction. The value lies in situations where low-latency access to external systems is extremely valuable. Consider as an example a contract trading with a market-maker. The contract sends a request to trade to the market-maker, who responds with an offered price. On a traditional blockchain, that response (a separate blockchain transaction) might be substantially delayed (by congestion on chain or by normal consensus or network broadcast delays, for example), and the contract has to be written to tolerate a fair amount of delay. This gives the market-maker a so-called "free option" to wait and receive more information about the price of the relevant assets before acting [18, 22, 38, 39]. Thruput, by contrast, can cut that delay down to a single network roundtrip, making that option as small as possible.

A contract could make queries to several market-makers, and trade with the best offered price quote. One could also query oracle data as needed within one transaction, instead of forcing the data provider to post an oracle update (for every possible data feed) in every block. A bank could even allow a transaction to query its KYC/AML processes, approving transfers of regulated assets without posting any private data on-chain.

11 Related Work

11.1 Semantic Changes

Thruput is a direct continuation of Groundhog [29]. That work is itself derived from SPEEDEX [30], which is a decentralized exchange in the same block of unordered, parallel transactions model. It operated an exchange by designing custom, commutative operations for transactions to interact with orderbooks. This approach is modeled on the Scalable Commutativity Rule of Clements et al. [10]. By redesigning kernel interfaces to be commutative, the authors build an operating system that scales extremely well. Other projects, including Canidio and Henneke [7], WalrasSwap [41], and Angstrom [34] settle batches of unordered trades (on sequential blockchains, like Ethereum), albeit for economic and market efficiency reasons, not performance.

Conflict-Free Replicated Data Types [33] are commutative interfaces for modifying shared (distributed) data structures. CRDTs, however, require that any update that is arbitrarily delayed can be reconciled with a local copy of the state. CRDTs can therefore implement an increasing counter, for example, but cannot implement a notion of a token balance, which must increase

[7] We do not want Thruput to act as a DoS amplification vector on other services.

and decrease and stay nonnegative, without some kind of synchronization. Two replicas might both subtract the last dollar from an account, and the CRDT must reconcile these updates. Balegas et al. [5] split the notion of a balance across a fixed, known set of replicas, and then each replica only admits subtraction to the shared counter that it locally can guarantee will not violate the nonnegativity constraint. That approach requires every replica to be trusted, and is akin to sharding a balance over many separate networks.

Garamvölgyi et al. [16] adds a "commutative add" operation to the Ethereum Virtual Machine, which lets two transactions that just increment a shared counter not conflict under optimistic concurrency control. Mitenkov et al. [24] construct "Deferred Object" types (deployed on the Aptos blockchain [23]) which modify highly contended items, like counters, in a structured way (i.e. output a counter increment without observing the value). This technique can reduce the frequency of transaction conflicts in optimistic concurrency control on some workloads, but does not guarantee parallelism on potentially adversarial workloads and still pays the overhead of concurrency control.

11.2 Parallelizing Sequential Execution

Accelerating operations on a sequential state machine is a long-running open systems and engineering research problem. Block-STM [17] optimistically executes transactions concurrently on many CPU cores while recording which objects are read and written to, retrying transactions when a speculatively read object changes. This kind of approach scales well when few transactions touch shared state, but cannot perform significantly better than sequential execution under high contention. Furthermore, the overhead of the concurrency control data structures introduces engineering complexity and adds significant performance overhead, especially when using many CPU cores.

Chen et al. [9] speculatively execute Ethereum transactions, and Lin et al. [20] instrument transactions so that an optimistic concurrency control process can retry only parts of a transaction, instead of the entire transaction, in case of conflicts. Zhang et al. [42] combines speculated execution traces with a branch predictor, so that actual execution results are more likely to match a pre-executed trace.

Eve [19] schedules batches of transactions in parallel using a conflict checker. Eve guesses at what transactions do not conflict with each other, executes batches of hopefully-nonconflicting transactions on separate CPUs, and then verifies that no conflicts occurred. Performance relies on this conflict checker; conflict check errors degrade performance (to no better than sequential execution, in the extreme). It is not clear if it is possible to write a conflict checker that is hardened against adversarial smart contract inputs without requiring transactions to specify exactly which keys are read from and written to. Thruput, by contrast, requires no predeclared read/write sets and no conflict checks, even on malicious inputs.

Strife [28] splits transactions into nonconflicting clusters, while Bohm [15] assigns a serialization order and only begins executing transactions when all read dependencies are available. Strife requires advance knowledge of read/write sets, while Bohm requires predeclared (or speculated) write sets.

Similar in spirit to Thruput are Aria [21] and LiTM [37], which execute batches of transactions in parallel optimistically, and then retry any that abort due to concurrency conflicts in a subsequent batch. These systems still maintain the abstraction of serializability.

Basil [36] allows clients to transact with a replicated key-value store using an optimistic locking protocol, so that neither malicious clients nor malicious nodes can halt the system. This system does not, however, support smart contracts or guarantee correctness of values written by clients.

Some projects use read-write sets [6, 40] or static analysis [8] to precompute sets of transactions to execute in parallel, include an observed dependency graph in a block proposal [3, 4, 13], or

change transaction order to increase parallelism [32], or often combinations of these techniques. Pan et al. [27] adds instruction-level parallelism to accelerate execution of individual transactions and designs an ASIC for Ethereum bytecode.

11.3 Sharding and Layer-2 Chains

12 Roadmap

References

[1] Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 Core. https://app.uniswap.org/whitepaper.pdf, 2020.

[2] Elvira Albert, Samir Genaim, Daniel Kirchner, and Enrique Martin-Martin. Formally verified EVM block-optimizations. In *International Conference on Computer Aided Verification*, pages 176–189. Springer, 2023.

[3] Parwat Singh Anjana, Sweta Kumari, Sathya Peri, Sachin Rathor, and Archit Somani. An efficient framework for optimistic concurrent execution of smart contracts. In *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, pages 83–92. IEEE, 2019.

[4] Parwat Singh Anjana, Hagit Attiya, Sweta Kumari, Sathya Peri, and Archit Somani. Efficient concurrent execution of smart contracts in blockchains using object-based transactional memory. In *International Conference on Networked Systems*, pages 77–93. Springer, 2020.

[5] Valter Balegas, Diogo Serra, Sérgio Duarte, Carla Ferreira, Marc Shapiro, Rodrigo Rodrigues, and Nuno Preguiça. Extending eventually consistent cloud databases for enforcing numeric invariants. In *2015 IEEE 34th Symposium on Reliable Distributed Systems (SRDS)*, pages 31–36. IEEE, 2015.

[6] Massimo Bartoletti, Letterio Galletta, and Maurizio Murgia. A true concurrent model of smart contracts executions. In *International Conference on Coordination Languages and Models*, pages 243–260. Springer, 2020.

[7] Andrea Canidio and Felix Henneke. Fair combinatorial auctions: Endogenous best execution in blockchain trade-intent markets, 2026. URL https://arxiv.org/abs/2408.12225.

[8] Atefeh Zareh Chahoki, Maurice Herlihy, and Marco Roveri. Conthereum: Concurrent ethereum optimized transaction scheduling for multi-core execution. In *2025 7th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*, pages 1–10. IEEE, 2025.

[9] Yang Chen, Zhongxin Guo, Runhuai Li, Shuo Chen, Lidong Zhou, Yajin Zhou, and Xian Zhang. Forerunner: Constraint-based speculative transaction execution for ethereum. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 570–587, 2021.

[10] Austin T Clements, M Frans Kaashoek, Nickolai Zeldovich, Robert T Morris, and Eddie Kohler. The scalable commutativity rule: Designing scalable software for multicore processors. *ACM Transactions on Computer Systems (TOCS)*, 32 (4):1–47, 2015.

[11] Compound. Comet: Interest accrual implementation. https://github.com/compound-finance/comet/blob/f766f51583c23acc33b2a7824654ef2029a96804/contracts/CometWithExtendedAssetList.sol#L243-L255. Function `accrueInternal`, revision `f766f51583c2`.

[12] Compound. Compound III docs. https://docs.compound.xyz/, 2022.

[13] Thomas Dickerson, Paul Gazzillo, Maurice Herlihy, and Eric Koskinen. Adding concurrency to smart contracts. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*, pages 303–312, 2017.

[14] Leonardo Donatacci. How to write Solana programs with sBPF assembly. https://www.helius.dev/blog/sbpf-assembly, September 2025. Published 4 September 2025. Accessed: 2026-10-18. Archived at https://web.archive.org/web/20261018203809/https://www.helius.dev/blog/sbpf-assembly.

[15] Jose M Faleiro and Daniel J Abadi. Rethinking serializable multiversion concurrency control. *arXiv preprint arXiv:1412.2324*, 2014.

[16] Péter Garamvölgyi, Yuxi Liu, Dong Zhou, Fan Long, and Ming Wu. Utilizing parallelism in smart contracts on decentralized blockchains by taming application-inherent conflicts. In *Proceedings of the 44th international conference on software engineering*, pages 2315–2326, 2022.

[17] Rati Gelashvili, Alexander Spiegelman, Zhuolun Xiang, George Danezis, Zekun Li, Dahlia Malkhi, Yu Xia, and Runtian Zhou. Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 232–244, 2023.

[18] Runchao Han, Haoyu Lin, and Jiangshan Yu. On the optionality and fairness of atomic swaps. In *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, pages 62–75, 2019.

[19] Manos Kapritsos, Yang Wang, Vivien Quema, Allen Clement, Lorenzo Alvisi, and Mike Dahlin. All about eve:{Execute-Verify} replication for {Multi-Core} servers. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*, pages 237–250, 2012.

[20] Haoran Lin, Hang Feng, Yajin Zhou, and Lei Wu. Parallelevm: Operation-level concurrent transaction execution for evm-compatible blockchains. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 211–225, 2025.

[21] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. Aria: a fast and practical deterministic oltp database. 2020.

[22] Bruno Mazorra, Burak Öz, Christoph Schlegel, and Fei Wu. The free option problem of ePBS. *arXiv preprint arXiv:2509.24849*, 2025.

[23] George Mitenkov, Satya Vusirikala, Rati Gelashvili, and Igor Kabiljo. AIP-47: Aggregators V2. https://github.com/aptos-foundation/AIPs/blob/main/aips/aip-047-aggregators-v2.md, August 2023. Status: Accepted. Standard (Core/Framework). Accessed: 2026-10-20.

[24] George Mitenkov, Igor Kabiljo, Zekun Li, Alexander Spiegelman, Satyanarayana Vusirikala, Zhuolun Xiang, Aleksandar Zlateski, Nuno P Lopes, and Rati Gelashvili. Deferred objects to enhance smart contract programming with optimistic parallel execution. *arXiv preprint arXiv:2405.06117*, 2024.

[25] Stephen Monn and Bikem Bengisu. ERC-7521: General intents for smart contract wallets. https://eips.ethereum.org/EIPS/eip-7521, September 2023. Draft, Standards Track: ERC. Created 2023-09-19. Accessed: 2026-10-19.

[26] Milad Nasr and Nicholas Carlini. Cryptanalysis of 7-round aes via the algebraic structure of its s-box. Technical report, Technical Report, Anthropic, 2026.

[27] Rui Pan, Chubo Liu, Guoqing Xiao, Mingxing Duan, Keqin Li, and Kenli Li. An algorithm and architecture co-design for accelerating smart contracts in blockchain. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–13, 2023.

[28] Guna Prasaad, Alvin Cheung, and Dan Suciu. Handling highly contended oltp workloads using fast dynamic partitioning. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 527–542, 2020.

[29] Geoffrey Ramseyer and David Mazières. Groundhog: Linearly-scalable smart contracting via commutative transaction semantics. arXiv preprint arXiv:2404.03201, 2024. URL https://arxiv.org/abs/2404.03201.

[30] Geoffrey Ramseyer, Ashish Goel, and David Mazières. SPEEDEX: A scalable, parallelizable, and economically efficient decentralized EXchange. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 849–875, 2023.

[31] RareSkills. The RareSkills book of Solidity gas optimization: 80+ tips. https://rareskills.io/post/gas-optimization, 2023. Accessed: 2026-10-18. Archived at https://web.archive.org/web/20260421224512/https://rareskills.io/post/gas-optimization.

[32] Pingcheng Ruan, Dumitrel Loghin, Quang-Trung Ta, Meihui Zhang, Gang Chen, and Beng Chin Ooi. A transactional perspective on execute-order-validate blockchains. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 543–557, 2020.

[33] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Symposium on Self-Stabilizing Systems*, pages 386–400. Springer, 2011.

[34] Karthik Srinivasan, Ludwig Thouvenin, and Ciamac Moallemi. Angstrom v1 whitepaper. Technical report, Sorella Labs, July 2025. URL https://app.angstrom.xyz/whitepaper-v1.pdf. Accessed: 2026-10-19. Archived at https://web.archive.org/web/20260306065539/https://app.angstrom.xyz/whitepaper-v1.pdf.

[35] Zygimantas Straznickas and Stephen A Weis. Hawk-n key recovery reduces to svp in dimension $n/2 + 1$. *Cryptology ePrint Archive*, 2026.

[36] Florian Suri-Payer, Matthew Burke, Zheng Wang, Yunhao Zhang, Lorenzo Alvisi, and Natacha Crooks. Basil: Breaking up bft with acid (transactions). In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 1–17, 2021.

[37] Yu Xia, Xiangyao Yu, William Moses, Julian Shun, and Srinivas Devadas. Litm: a lightweight deterministic software transactional memory system. In *Proceedings of the 10th International Workshop on Programming Models and Applications for Multicores and Manycores*, pages 1–10, 2019.

[38] Jiahua Xu, Damien Ackerer, and Alevtina Dubovitskaya. A game-theoretic analysis of cross-chain atomic swaps with HTLCs. In *2021 IEEE 41st international conference on distributed computing systems (ICDCS)*, pages 584–594. IEEE, 2021.

[39] Yingjie Xue and Maurice Herlihy. Hedging against sore loser attacks in cross-chain transactions. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, pages 155–164, 2021.

[40] Wei Yu, Kan Luo, Yi Ding, Guang You, and Kai Hu. A parallel smart contract model. In *Proceedings of the 2018 International Conference on Machine Learning and Machine Intelligence*, pages 72–77, 2018.

[41] Sergio A. Yuhjtman. Walraswap: a solution to uniform price batch auctions, 2023. URL https://arxiv.org/abs/2310.12255.

[42] Shijie Zhang, Ru Cheng, Xinpeng Liu, Jiang Xiao, Hai Jin, and Bo Li. Seer: Accelerating blockchain transaction execution by fine-grained branch prediction. *Proceedings of the VLDB Endowment*, 18(3):822–835, 2024.